LABORATORIO AA. 2025/2026

Computazione Quantistica INF0427

Studio sperimentale dell'algoritmo di Grover

Pierluigi Boscaglia

Matricola: 983553

1 Introduzione

In questo progetto studio l'algoritmo di Grover (paper originale), che permette di cercare una soluzione in uno spazio non strutturato mediante l'amplificazione delle ampiezze. L'obiettivo degli esperimenti è capire come cambia la probabilità di successo al variare del numero di iterazioni e quanto il comportamento ideale si discosta da quello osservato in presenza di rumore.

Il percorso parte dalla costruzione visuale dei circuiti con Quantum Programming Studio e IBM Quantum Composer. Prosegue con piccoli esercizi in Python usando Cirq di Google Quantum AI, per seguire passo dopo passo l'evoluzione delle ampiezze e confrontare le probabilità ideali con le frequenze ottenute dalle misure simulate. Infine utilizzo Qiskit per approfondire le simulazioni ideali e con rumore e svolgere prove su hardware reale tramite IBM Quantum Platform.

2 Primi esperimenti con strumenti visuali

Per iniziare ho costruito due piccoli circuiti attraverso strumenti di composizione visuale: il primo con Quantum Programming Studio, il secondo con IBM Quantum Composer. Ho inserito le porte direttamente sulle linee dei qubit, seguendo da sinistra a destra le operazioni di preparazione, oracolo, diffusore e misura. Questo approccio mi ha permesso di ragionare sulle singole operazioni prima di passare alla costruzione dei circuiti in Python.

Entrambi gli esempi utilizzano due qubit di ricerca, quindi quattro stati possibili. Le porte Hadamard iniziali preparano la sovrapposizione uniforme:

$$|s\rangle = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2}.$$

Inizialmente, ciascuno dei quattro stati ha probabilità 1/4 di essere misurato. L'oracolo inverte il segno dell'ampiezza dello stato cercato, senza modificarne la probabilità; il diffusore sfrutta questa differenza di fase per amplificarne l'ampiezza. In entrambi i circuiti ho applicato una sola iterazione di Grover, composta da oracolo e diffusore. Nel seguito rappresento gli stati nell'ordine $|q_1 q_0\rangle$: il qubit q_0, disegnato in alto nel circuito, corrisponde al bit più a destra nella stringa. Mantengo questa convenzione in tutta la relazione per rendere univoca la lettura degli stati.

2.1 Quantum Programming Studio: circuito senza ancilla

Nel primo circuito ho utilizzato soltanto i due qubit di ricerca. L'oracolo è costruito con porte X, Hadamard e una CNOT. Una Hadamard sul target prima e dopo la CNOT permette di ottenere una porta controlled-Z:

$$H_{q_1}\,\mathrm{CNOT}_{q_0\to q_1}\,H_{q_1} = \mathrm{CZ},$$

dove q_0 è il controllo della CNOT e q_1 il suo target. La CZ lascia invariati gli stati della base, tranne $|11\rangle$, di cui inverte il segno dell'ampiezza. Per marcare invece $|10\rangle$, applico una porta X su q_0 prima e dopo questo blocco. La prima X trasforma temporaneamente $|10\rangle$ in $|11\rangle$; la CZ introduce il segno meno e la seconda X ripristina lo stato iniziale:

$$|10\rangle \xrightarrow{X_{q_0}} |11\rangle \xrightarrow{\mathrm{CZ}} -|11\rangle \xrightarrow{X_{q_0}} -|10\rangle.$$

Gli altri stati vengono ripristinati senza cambiamenti di segno, perché la prima X non li porta in $|11\rangle$.

> L'effetto complessivo dell'oracolo è quindi **invertire soltanto il segno dell'ampiezza** della soluzione $|10\rangle$, **senza modificarne i bit o la probabilità di misura**.

Dopo l'oracolo, il diffusore applica H e X a entrambi i qubit, poi una seconda CZ, ancora realizzata con H–CNOT–H sul target, e infine annulla le X e applica le Hadamard finali. Il qubit target delle CNOT fa parte del registro di ricerca: non viene aggiunto un qubit ausiliario.

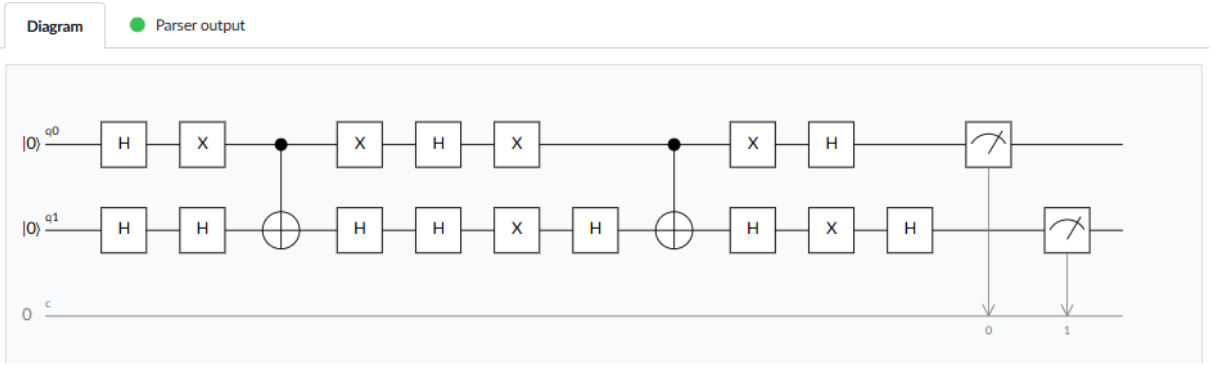


Figura 1: Circuito realizzato in Quantum Programming Studio con due qubit di ricerca, senza ancilla.



Figura 2: Risultato visualizzato in Quantum Programming Studio: $q_0 = 0$ e $q_1 = 1$, corrispondenti allo stato $|10\rangle$ nella convenzione adottata.

2.2 Quantum Composer: circuito con ancilla

Nel secondo circuito ho mantenuto due qubit di ricerca e aggiunto un terzo qubit, indicato con `anc[0]` nello screenshot. Questo qubit non rappresenta un ulteriore bit della soluzione: lo spazio di ricerca rimane di quattro stati. Le porte X e H preparano l'ancilla nello stato

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

L'oracolo è una Toffoli con i due qubit di ricerca come controlli e l'ancilla come target. Poiché $X|-\rangle = -|-\rangle$, quando entrambi i controlli valgono uno si introduce un segno meno nell'ampiezza corrispondente. Nel *phase kickback* l'ancilla permette di modificare la fase dello stato $|11\rangle$, pur rimanendo nello stato $|-\rangle$.

Per costruire il diffusore applico H e X sui qubit di ricerca, riutilizzo la Toffoli con la stessa ancilla, quindi applico nuovamente X e H. Alla fine misuro soltanto i due qubit di ricerca. In Figura 3, l'ancilla è la linea `anc[0]`.

Perché uso una porta X sull'ancilla?
Nello schema mostrato a lezione, il cambio di segno viene ottenuto con una porta Z controllata, applicata direttamente ai qubit di ricerca. Nel mio circuito ottengo lo stesso effetto con una Toffoli e un qubit aggiuntivo: l'ancilla. Per far funzionare questo meccanismo, preparo l'ancilla in $|-\rangle$ applicando prima X e poi H. Quando i due controlli valgono uno, la Toffoli applica una X all'ancilla: poiché $X|-\rangle = -|-\rangle$, cambia il segno dell'ampiezza corrispondente nel registro di ricerca. Le porte H e X sui qubit di ricerca rimangono necessarie per completare il diffusore. I due modi di costruirlo producono le stesse probabilità di misura

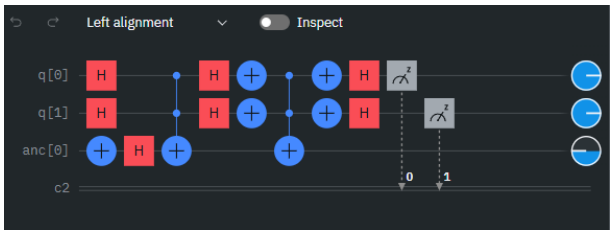


Figura 3: Circuito realizzato in IBM Quantum Composer: due qubit di ricerca e un'ancilla per il phase kickback.

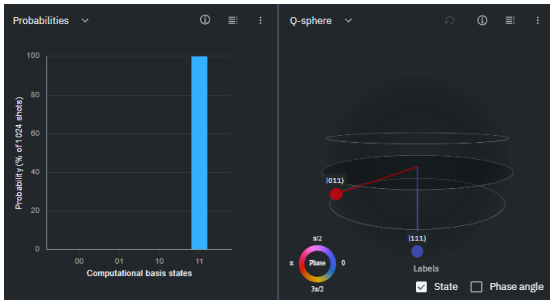


Figura 4: Risultato in Quantum Composer: il registro di ricerca viene misurato nello stato $|11\rangle$. La Q-sphere include anche l'ancilla.

2.3 Confronto e funzionamento dei due circuiti

Le due costruzioni realizzano la stessa idea di Grover con risorse diverse. Il primo circuito modifica direttamente le fasi del registro attraverso una CZ (decomposta come visto); il secondo ottiene la modifica di fase usando un'ancilla. Questa differenza dipende dalla costruzione scelta, non da un obbligo imposto dai due strumenti grafici. Inoltre i target sono diversi: $|10\rangle$ nel primo circuito e $|11\rangle$ nel secondo.

Dopo l'oracolo l'ampiezza della soluzione vale $-1/2$ e le altre valgono $1/2$. La loro media è $1/4$: il diffusore standard trasforma ogni ampiezza a in $2\overline{a} - a$, portando quella della soluzione a 1 e le altre a 0. Le sequenze mostrate realizzano il diffusore a meno di un segno globale, che non modifica le probabilità di misura.

3 Esercizio svolto con Cirq di Google Quantum AI

Dopo i circuiti costruiti con gli strumenti visuali, ho realizzato un piccolo esperimento in Python con Cirq di Google Quantum AI. Come Qiskit, Cirq è una libreria Python; una sua particolarità è organizzare esplicitamente il circuito in momenti, raggruppando le operazioni su qubit distinti: una struttura comoda per seguire la simulazione passo dopo passo. Il codice viene eseguito su un simulatore ideale locale, senza collegarsi a un computer quantistico. Ho scelto tre qubit di ricerca, quindi otto stati possibili, e una sola soluzione da trovare: $|101\rangle$, nell'ordine $|q_2 q_1 q_0\rangle$.

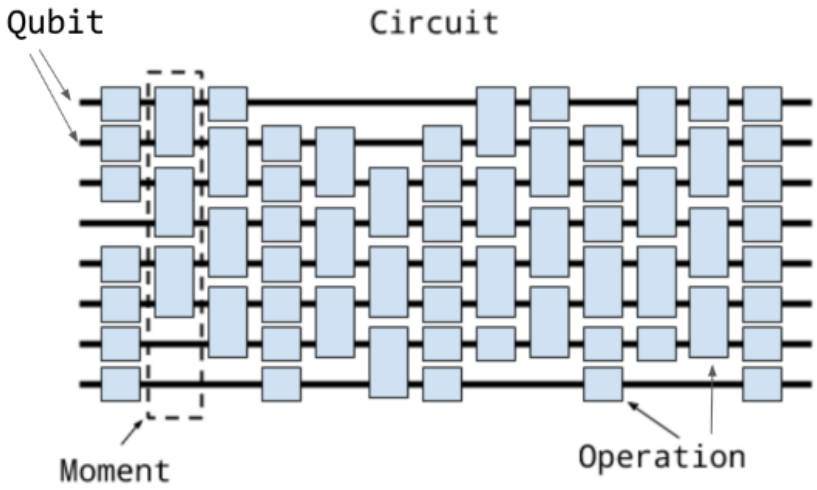


Figura 5: Organizzazione di un circuito in Cirq: qubit, operazioni e momenti.

Costruzione del circuito. Ho mantenuto la costruzione con ancilla usata in Composer: il circuito contiene quindi quattro qubit totali, ma solo tre codificano la soluzione. Per selezionare $|101\rangle$, invece di $|111\rangle$, inserisco una X sul qubit centrale q_1 prima e dopo questa porta. Il phase kickback cambia così il segno della sola ampiezza cercata. Il diffusore riutilizza la stessa ancilla, con le porte H e X sui qubit di ricerca prima del blocco controllato e X e H dopo.

Osservazione dei passaggi. Nel notebook stampo il circuito durante la costruzione e seguo la simulazione dopo ogni *momento*, cioè un gruppo di operazioni su qubit distinti. Per ogni stato di ricerca mostro le due ampiezze associate ai valori zero e uno dell'ancilla. La probabilità del registro di ricerca si ottiene sommando i moduli al quadrato di queste due ampiezze. Queste informazioni sono disponibili nel simulatore: non sono misure intermedie inserite nel circuito. Le stampe permettono di osservare che l'oracolo completo cambia il segno delle ampiezze senza cambiare le probabilità, mentre il diffusore aumenta quella della soluzione. Verifico inoltre che, dopo la preparazione, l'ancilla rimanga nello stato $|-\rangle$.

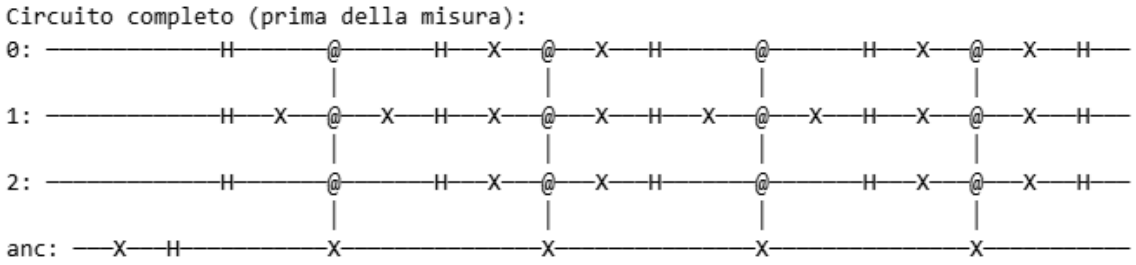


Figura 6: Circuito di Grover costruito con Cirq prima della misura: tre qubit di ricerca, un'ancilla e due iterazioni.

Momento 3: Sovrapposizione iniziale

Ricerca	Amp(anc=0)	Amp(anc=1)	Prob. ricerca
000	+0.25000	-0.25000	12.500%
001	+0.25000	-0.25000	12.500%
010	+0.25000	-0.25000	12.500%
011	+0.25000	-0.25000	12.500%
100	+0.25000	-0.25000	12.500%
101	+0.25000	-0.25000	12.500%
110	+0.25000	-0.25000	12.500%
111	+0.25000	-0.25000	12.500%

>>> Fine Sovrapposizione: P(101) = 12.50000%

Momento 3

Momento 6: Oracolo: ripristino del bit centrale

Ricerca	Amp(anc=0)	Amp(anc=1)	Prob. ricerca
000	+0.25000	-0.25000	12.500%
001	+0.25000	-0.25000	12.500%
010	+0.25000	-0.25000	12.500%
011	+0.25000	-0.25000	12.500%
100	+0.25000	-0.25000	12.500%
101	-0.25000	+0.25000	12.500%
110	+0.25000	-0.25000	12.500%
111	+0.25000	-0.25000	12.500%

>>> Fine Oracolo 1: P(101) = 12.50000%

Momento 6

Momento 11: Diffusore: H

Ricerca	Amp(anc=0)	Amp(anc=1)	Prob. ricerca
000	-0.12500	+0.12500	3.125%
001	-0.12500	+0.12500	3.125%
010	-0.12500	+0.12500	3.125%
011	-0.12500	+0.12500	3.125%
100	-0.12500	+0.12500	3.125%
101	-0.62500	+0.62500	78.125%
110	-0.12500	+0.12500	3.125%
111	-0.12500	+0.12500	3.125%

>>> Fine Diffusore 1: P(101) = 78.12500%

Momento 11

Momento 19: Diffusore: H

Ricerca	Amp(anc=0)	Amp(anc=1)	Prob. ricerca
000	-0.06250	+0.06250	0.781%
001	-0.06250	+0.06250	0.781%
010	-0.06250	+0.06250	0.781%
011	-0.06250	+0.06250	0.781%
100	-0.06250	+0.06250	0.781%
101	+0.68750	-0.68750	94.531%
110	-0.06250	+0.06250	0.781%
111	-0.06250	+0.06250	0.781%

>>> Fine Diffusore 2: P(101) = 94.53125%

Momento 19

Figura 7: Stampe della simulazione in quattro momenti del circuito di Grover.

Risultati. Ho applicato due iterazioni di Grover. I grafici del notebook mostrano come cambia la distribuzione sugli otto stati; per la soluzione si ottiene:

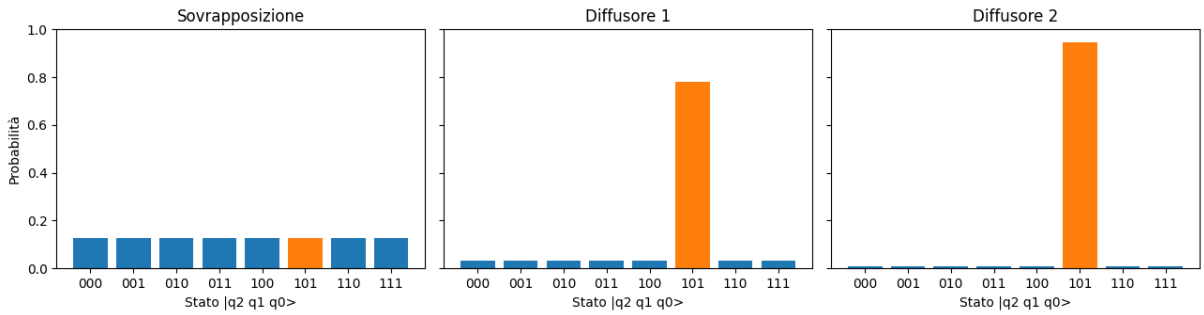


Figura 8: Distribuzione delle probabilità nella simulazione con Cirq: sovrapposizione iniziale e risultati dopo il primo e il secondo diffusore. Lo stato cercato $|101\rangle$ è evidenziato in arancione.

Il risultato coincide con la previsione teorica $P_k = \sin^2((2k+1)\theta)$, con $\theta = \arcsin(1/\sqrt{8})$. Infine ho misurato soltanto i tre qubit di ricerca per 1024 ripetizioni: nell'esecuzione salvata nel notebook, 101 compare 964 volte, con una frequenza di circa 94,14%.

Stato	Conteggi	Frequenza	Probabilità ideale
000	4	0.39%	0.78%
001	9	0.88%	0.78%
010	9	0.88%	0.78%
011	13	1.27%	0.78%
100	10	0.98%	0.78%
101	964	94.14%	94.53%
110	11	1.07%	0.78%
111	4	0.39%	0.78%

Figura 9: Risultati di 1024 shot nella simulazione con Cirq.

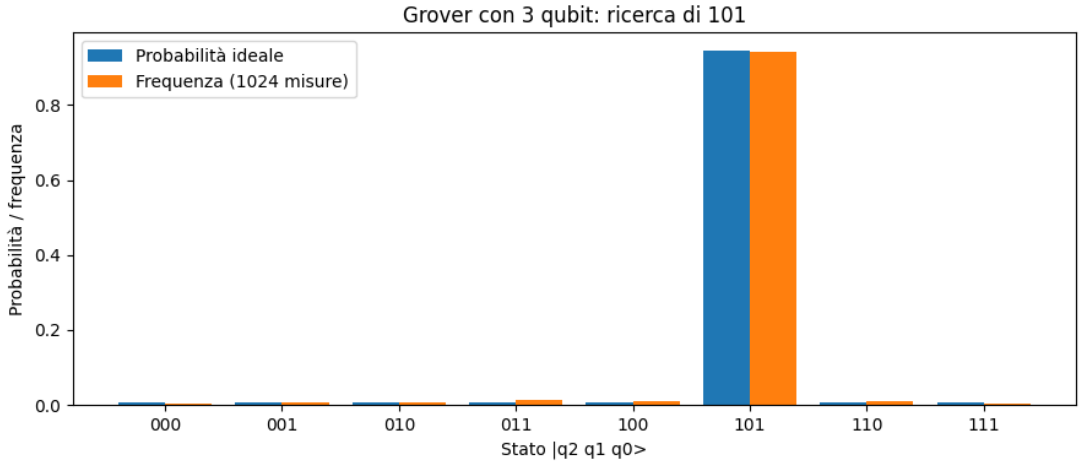


Figura 10: Confronto grafico tra le probabilità.

4 Studio Sperimentale con Qiskit di IBM Quantum e hardware reale

Dopo i primi esercizi, ho approfondito lo studio di Grover nel notebook `Grover.ipynb`, utilizzando Qiskit. L'obiettivo è verificare il comportamento previsto dalla teoria e capire come cambiano i risultati quando si passa dalla simulazione ideale a condizioni più vicine a quelle di un dispositivo reale.

Ho organizzato gli esperimenti in tre modalità:

- simulazione ideale,
- simulazione con rumore ricavato dai profili dei dispositivi IBM (*fake backend*),
- esecuzione su hardware quantistico IBM.

Nella simulazione ideale confronto le probabilità ricavate dal vettore di stato con le frequenze osservate nelle misure, per mettere in evidenza le fluttuazioni dovute al numero finito di ripetizioni. Studio inoltre il caso in cui il numero di soluzioni non sia noto, scegliendo casualmente quante iterazioni eseguire e verifico se il risultato è una soluzione valida. Passo poi alle simulazioni con rumore e alle prove su hardware reale, per osservare come gli errori delle porte e della lettura modifichino la probabilità di successo rispetto al caso ideale.

4.1 Ambiente di lavoro e strumenti utilizzati

- **Python, ambiente virtuale e notebook in Visual Studio Code.** Il codice viene eseguito nell'ambiente locale `.venv`, con un kernel Python per i notebook Jupyter. Questo formato permette di affiancare codice, spiegazioni, grafici e risultati. Le dipendenze del progetto sono elencate in `requirements.txt`.
  notebook Jupyter in VS Code.

- **Qiskit: costruzione e verifica dei circuiti.** Uso `QuantumCircuit` per costruire l'oracolo, il diffusore e il circuito completo, inserendo esplicitamente le porte e le misure. Le classi `Operator` e `Statevector` permettono di controllare gli operatori e calcolare ampiezze e probabilità ideali.
  documentazione Qiskit.

- **Transpiler di Qiskit.** Uso `transpile` e il generatore di pass manager `generate_preset_pass_manager` per adattare i circuiti alle porte e ai collegamenti disponibili sul backend scelto. Questo passaggio è necessario soprattutto prima dell'esecuzione sull'hardware reale.
  transpiler di Qiskit.

- **Qiskit Aer.** La classe `AerSimulator` esegue localmente i circuiti con misure ripetute, sia nel caso ideale sia in presenza di rumore. Il metodo `from_backend` configura il simulatore a partire dalle caratteristiche di un backend.
  AerSimulator.

- **Fake backend IBM.** Uso `FakeManilaV2`, `FakeNairobiV2` e `FakeMumbaiV2`, importati da `qiskit_ibm_runtime.fake_provider`, per impostare le simulazioni con rumore. Questi profili descrivono caratteristiche ed errori di dispositivi IBM; le prove rimangono simulazioni locali e non riproducono necessariamente le condizioni attuali delle QPU.
  fake provider IBM.

- **IBM Quantum Platform e Qiskit IBM Runtime.** Accedo all'hardware tramite il mio account IBM. `QiskitRuntimeService` gestisce la connessione, la scelta del backend e il recupero dei job; `SamplerV2` invia i circuiti e restituisce i risultati delle misure. Salvo i risultati per poterli analizzare anche senza ripetere l'esecuzione sulla QPU.
  API di Qiskit IBM Runtime.

4.2 Descrizione e implementazione dell'algoritmo

L'algoritmo di Grover cerca una soluzione tra $N = 2^n$ stati, rappresentati da n qubit. Nel notebook indico con W l'insieme delle soluzioni e con M il loro numero. Per costruire e verificare gli esperimenti scelgo esplicitamente le stringhe da marcare: non carico un database classico, ma realizzo un oracolo che riconosce gli stati scelti. Le stringhe seguono la convenzione $|q_{n-1} \dots q_0\rangle$.

Funzionamento. Il circuito parte dallo stato $|0 \dots 0\rangle$. Una porta Hadamard su ciascun qubit prepara la sovrapposizione uniforme

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle.$$

Ogni iterazione applica poi due operazioni:

1. **Oracolo.** Cambia il segno dell'ampiezza degli stati appartenenti a W, lasciando invariati gli altri:
$$O_W|x\rangle = \begin{cases} -|x\rangle & \text{se } x \in W, \\ |x\rangle & \text{altrimenti.} \end{cases}$$
Questo passaggio modifica le fasi, ma non le probabilità di misura.

2. **Diffusore.** Applica la riflessione $D = 2|s\rangle\langle s| - I$, che trasforma ogni ampiezza a_x in $2\overline{a} - a_x$, dove $\overline{a}$ è la media delle ampiezze. Insieme all'oracolo, permette di amplificare le soluzioni.

Dopo k iterazioni lo stato è $(DO_W)^k|s\rangle$ e la misura restituisce una stringa candidata. Nel caso ideale la probabilità complessiva di ottenere una soluzione è

$$P(k) = \sin^2((2k+1)\theta), \qquad \theta = \arcsin\sqrt{\frac{M}{N}}.$$

Codice sorgente e scelte implementative. Il codice è scritto in Python con Qiskit e si trova nelle sezioni 5–8 di `Grover.ipynb`. Le funzioni principali sono:

- `build_phase_oracle`: per ogni soluzione applica X ai qubit corrispondenti ai bit zero, una Z multicontrollata e infine applica nuovamente le stesse X per ripristinare i bit. Il cambio di fase su $|11\ldots1\rangle$ viene così trasferito alla stringa desiderata.
- `build_diffuser`: usa `grover_operator` con un oracolo identità per ottenere soltanto il diffusore D. La fase globale necessaria al confronto delle ampiezze è già inclusa da Qiskit e non viene aggiunta nuovamente.
- `build_grover_circuit`: prepara la sovrapposizione uniforme con le Hadamard e, a ogni iterazione, applica prima l'oracolo e poi il diffusore. Ripete questo blocco il numero richiesto di volte e aggiunge, se richieste, le misure finali.

Materiale di riferimento. La costruzione segue l'algoritmo descritto nel materiale teorico del corso. Nel notebook sono inoltre richiamate le spiegazioni di IBM Quantum sull' interpretazione geometrica e sulla scelta delle iterazioni. Per il codice di oracolo e diffusore ho adottato una costruzione molto vicina agli esempi della documentazione IBM, adattandola alle funzioni e ai parametri utilizzati negli esperimenti del notebook:

- **Oracolo:** il tutorial IBM sull'algoritmo di Grover presenta la funzione `grover_oracle`, che applica X ai bit zero della soluzione, una Z multicontrollata e infine le stesse X. La funzione `build_phase_oracle` è un adattamento di questo codice: mantiene l'inversione della stringa per rispettare l'ordine dei qubit e usa la stessa porta `MCMTGate` con `ZGate`.
- **Diffusore:** la documentazione Qiskit di `grover_operator` viene utilizzata direttamente nella funzione `build_diffuser`. Poiché `grover_operator` costruisce il prodotto DO, passo come oracolo un circuito vuoto, che realizza l'identità: ottengo così soltanto D.

Il codice utilizza quindi esempi e strumenti forniti da IBM, evitando cosi costruzioni sviluppate da zero.

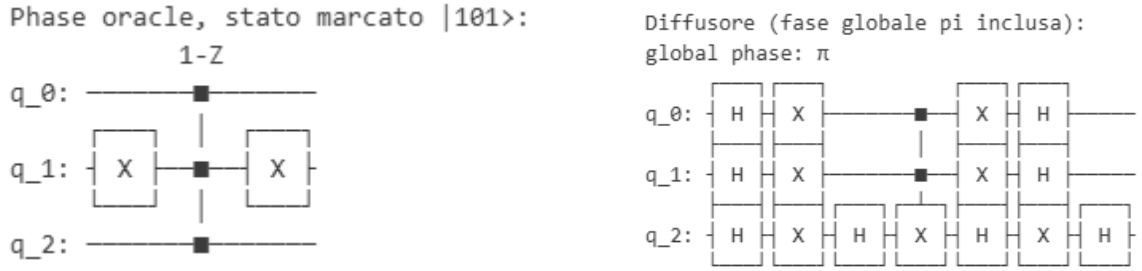


Figura 11: Oracolo di fase per lo stato $|101\rangle$ e diffusore costruiti con Qiskit su tre qubit di ricerca, senza ancilla dedicata.

Per verificare il funzionamento dei due circuiti, nella sezione 6.2 del notebook confronto le ampiezze degli otto stati prima dell'oracolo, dopo l'oracolo e dopo il diffusore, considerando una sola iterazione.

	basis_state	amplitude_before_oracle	amplitude_after_oracle	amplitude_after_diffuser
0	000	0.353553	0.353553	0.176777
1	001	0.353553	0.353553	0.176777
2	010	0.353553	0.353553	0.176777
3	011	0.353553	0.353553	0.176777
4	100	0.353553	0.353553	0.176777
5	101	0.353553	-0.353553	0.883883
6	110	0.353553	0.353553	0.176777
7	111	0.353553	0.353553	0.176777

Figura 12: Ampiezze degli otto stati prima dell'oracolo, dopo l'oracolo e dopo il diffusore, per una singola iterazione con soluzione $|101\rangle$.

Stima del numero di iterazioni. Nel punto 8 del notebook calcolo quante iterazioni eseguire per fermarmi vicino al primo massimo della probabilità di successo. Con $N = 2^n$ stati e M soluzioni, calcolo prima $\theta = \arcsin\sqrt{M/N}$ e poi

$$k_* = \frac{\pi}{4\theta} - \frac{1}{2}.$$

Il valore k_* è continuo: il codice confronta la probabilità $P(k) = \sin^2((2k+1)\theta)$ nei due interi non negativi vicini e sceglie quello con probabilità maggiore, preferendo il minore in caso di parità. Per poche soluzioni rispetto a N, uso l'approssimazione $\theta \simeq \sqrt{M/N}$, da cui

$$k_* \simeq \frac{\pi}{4}\sqrt{\frac{N}{M}} - \frac{1}{2}. \qquad \text{Se } M = 1: \quad k_* \simeq \frac{\pi}{4}\sqrt{N} - \frac{1}{2}.$$

La tabella confronta tre valori: `approximate_k` è la stima più semplice $(\pi/4)\sqrt{N/M}$, che trascura anche il termine $-1/2$; `first_continuous_maximum` è k_* calcolato con l'arcoseno; `chosen_k` è il numero intero scelto dal codice. Con una sola soluzione e 2, 3 e 4 qubit ottengo rispettivamente 1, 2 e 3 iterazioni.

	n	N	M	approximate_k	first_continuous_maximum	chosen_k
0	2	4	1	1.570796	1.000000	1
1	3	8	1	2.221441	1.673408	2
2	4	16	1	3.141593	2.608269	3

Figura 13: Confronto tra la stima approssimata, il primo massimo continuo e il numero di iterazioni scelto nel punto 8 del notebook.

4.3 Verifiche ideali

Nel capitolo 9 confronto teoria e simulazione senza rumore. Con `Statevector` verifico le probabilità per 2, 3 e 4 qubit e da 0 a 8 iterazioni, con tolleranza assoluta 10^{-12}. Osservo amplificazione e overshooting.

n	N	M	k	theoretical_probability	statevector_probability	absolute_error
2	4	1	0	0.250000	0.250000	1.110223e-16
2	4	1	1	1.000000	1.000000	1.332268e-15
2	4	1	2	0.250000	0.250000	1.110223e-16
2	4	1	3	0.250000	0.250000	8.881784e-16
2	4	1	4	1.000000	1.000000	4.218847e-15
2	4	1	5	0.250000	0.250000	9.436896e-16
2	4	1	6	0.250000	0.250000	2.275957e-15
2	4	1	7	1.000000	1.000000	7.105427e-15
2	4	1	8	0.250000	0.250000	1.887379e-15
3	8	1	0	0.125000	0.125000	8.326673e-17
3	8	1	1	0.781250	0.781250	1.776357e-15
3	8	1	2	0.945312	0.945312	3.885781e-15
3	8	1	3	0.330078	0.330078	1.831868e-15
3	8	1	4	0.012207	0.012207	2.185752e-16
3	8	1	5	0.547974	0.547974	5.773160e-15
3	8	1	6	0.999786	0.999786	1.154632e-14
3	8	1	7	0.576973	0.576973	7.216450e-15
3	8	1	8	0.019457	0.019457	2.116363e-16

Figura 14: Teoria e `Statevector`: estratto per 2 e 3 qubit.

Con `AerSimulator` eseguo poi 4096 shot per circuiti da 2 a 6 qubit, con iterazioni vicine al primo massimo. Confronto la frequenza (successi/shot) con la probabilità teorica: gli scarti sono dovuti al campionamento, anche in assenza di rumore.

Qubit	Iterazioni	Shots	Probabilità teorica	Frequenza misurata
2	0	4096	0.25	0.2461
2	1	4096	1.0	1.0
2	2	4096	0.25	0.2449
2	3	4096	0.25	0.2498
3	0	4096	0.125	0.1257
3	1	4096	0.7813	0.7874
3	2	4096	0.9453	0.9441
3	3	4096	0.3301	0.3279
3	4	4096	0.0122	0.0137

Figura 15: Misure simulate con 4096 shot: estratto per 2 e 3 qubit.

Infine ripeto quattro volte il circuito con 4 qubit e 3 iterazioni, con 4096 shot per prova e semi diversi, per osservare le fluttuazioni rispetto alla stessa probabilità teorica.

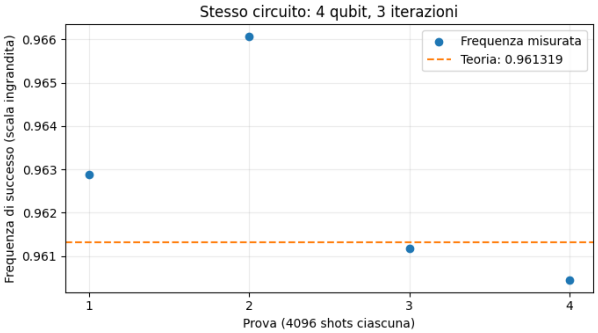


Figura 16: Quattro prove con 4 qubit, 3 iterazioni e 4096 shot ciascuna: confronto delle frequenze di successo con la probabilità teorica.

4.4 Dimensione del problema e costo del circuito

Nei capitoli 10 e 11 del notebook studio come cambiano le risorse necessarie al variare del numero di qubit e delle soluzioni. Le prove sono ancora ideali, con 4096 shot per circuito.

Aumento del numero di qubit. Nel capitolo 10 passo da 2 a 7 qubit, aumentando lo spazio di ricerca da 4 a 128 stati. Mantengo una sola soluzione, $|00\ldots01\rangle$, e scelgo per ogni dimensione il numero intero di iterazioni vicino al primo massimo teorico. La tabella e i grafici confrontano le iterazioni con la stima $(\pi/4)\sqrt{N}$ e riportano profondità, numero totale di porte e numero di porte a due qubit del circuito compilato nella base `u`, `cx`. Questo confronto distingue il numero di chiamate all'oracolo dal costo necessario a realizzarle: aumentando i qubit, anche una singola iterazione richiede una costruzione più articolata (questo anticipa già il problema che avremo con l'introduzione di rumore).

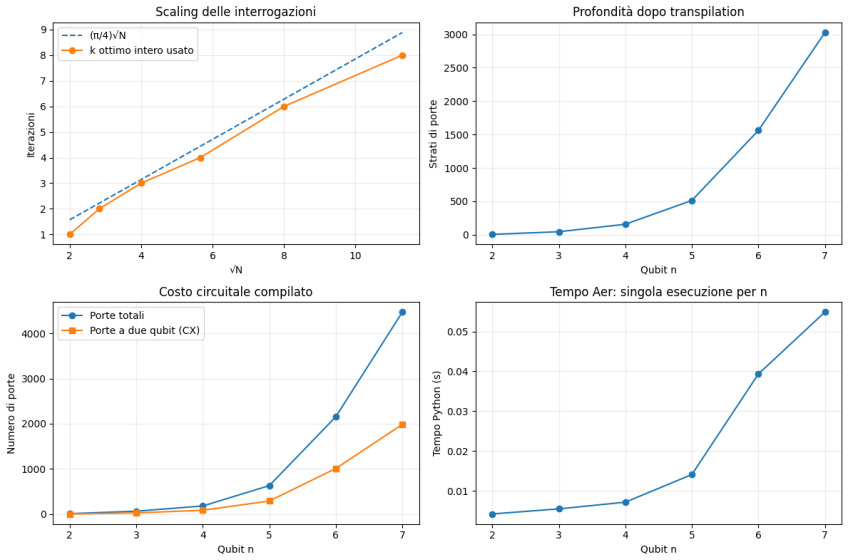


Figura 17: Scaling ideale: risorse del circuito al crescere del numero di qubit, dopo la compilazione nella base di porte `u` e `cx`.

Variazione del numero di soluzioni. Nel capitolo 11 fisso 5 qubit, quindi $N = 32$, e considero $M = 1, 2, 4, 8$ soluzioni, provando da 0 a 8 iterazioni. Il successo corrisponde alla misura di una qualsiasi delle stringhe marcate: sommo quindi i relativi conteggi prima di dividere per gli shot. Confronto le frequenze con $P(k) = \sin^2((2k+1)\theta)$, dove $\theta = \arcsin\sqrt{M/N}$. Aumentando le soluzioni, il primo massimo viene raggiunto con meno iterazioni. Questo non implica però che il circuito richieda sempre meno porte: nel mio oracolo ogni soluzione viene marcata separatamente, quindi il costo di una chiamata cresce con il numero di stati da riconoscere.

	M	theta	k_opt	theoretical_probability	measured_probability	standard_error
4	1	0.1777	4	0.9992	0.9995	0.0003
12	2	0.2527	3	0.9613	0.9648	0.0029
20	4	0.3614	2	0.9453	0.9429	0.0036
28	8	0.5236	1	1.0	1.0	0.0

Figura 18: Risultati per $N = 32$ e diversi numeri di soluzioni: iterazioni scelte, probabilità teoriche e frequenze misurate.

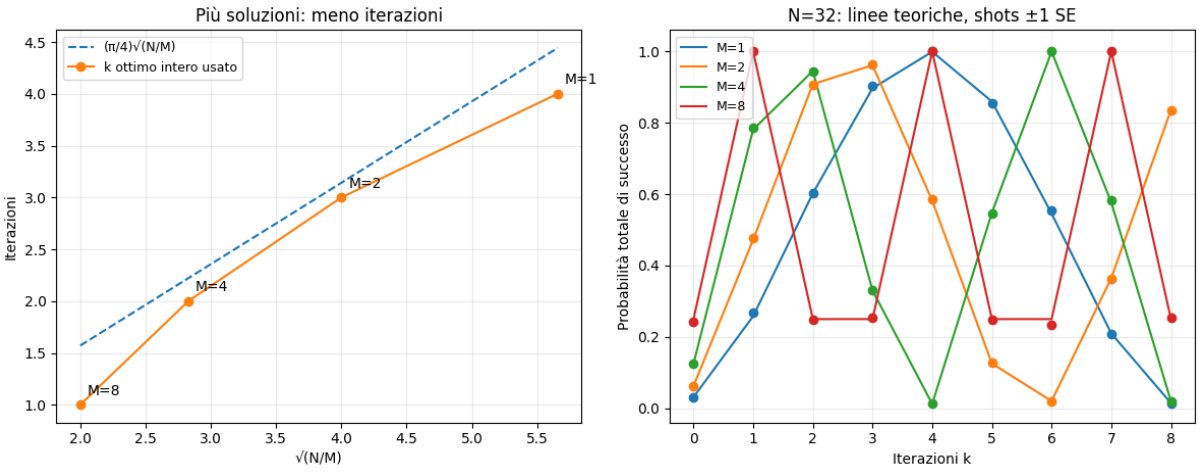


Figura 19: Iterazioni ottimali e probabilità di successo al variare del numero di soluzioni, con 4096 shot per circuito.

Numero di soluzioni ignoto. Quando il numero di soluzioni M non è noto, non posso calcolare direttamente il numero ottimale di iterazioni. Ripetere sempre lo stesso numero di iterazioni potrebbe far terminare ogni prova in un punto con bassa probabilità di successo. Il metodo BBHT, proposto da Boyer, Brassard, Hoyer e Tapp, affronta questo problema scegliendo casualmente quante iterazioni eseguire. A ogni tentativo preparo nuovamente il circuito, misuro un candidato e verifico classicamente se è una soluzione; in caso contrario amplio l'intervallo di scelta e riprovo.

La scelta casuale **non elimina le oscillazioni della probabilità**, ma evita di usare sempre un numero di iterazioni sfavorevole. Non serve trovare esattamente un picco: basta campionare anche valori per cui la probabilità di successo è sufficientemente alta. Se esistono soluzioni, ripetere i tentativi permette di aumentare la probabilità complessiva di trovarne una, senza conoscere M.

Nel capitolo 11 uso una variante di questa ricerca con un limite al numero di tentativi. Ripeto la ricerca 100 volte per ciascun valore di M, confrontando successi e chiamate all'oracolo. Il programma di ricerca non usa M per scegliere le iterazioni; le soluzioni sono note al protocollo sperimentale per costruire l'oracolo e il controllo dei candidati.

M	prove	successi	tentativi_medi	query_fase_medie	query_totali_medie	verifiche_classiche_attese
1	100	100	5.23	3.19	8.42	32.0
2	100	100	3.92	1.93	5.85	16.0
4	100	100	2.61	0.92	3.53	8.0
8	100	100	2.38	0.61	2.99	4.0

Figura 20: Riepilogo delle 100 ricerche randomizzate per ciascun valore di M: successi e costi medi.

	M	repetition	tentativo	finestra	k	candidato	successo
0	1	0	1	1.000	0	01001	False
1	1	0	2	1.200	0	00101	False
2	1	0	3	1.440	1	10001	False
3	1	0	4	1.728	0	00100	False
4	1	0	5	2.074	1	00101	False
5	1	0	6	2.488	0	11111	False
6	1	0	7	2.986	2	10011	False
7	1	0	8	3.583	3	00001	True

Figura 21: Tentativi della prima ricerca con $M = 1$: la soluzione viene trovata all'ottavo tentativo. La colonna `repetition=0` identifica la prima ricerca.

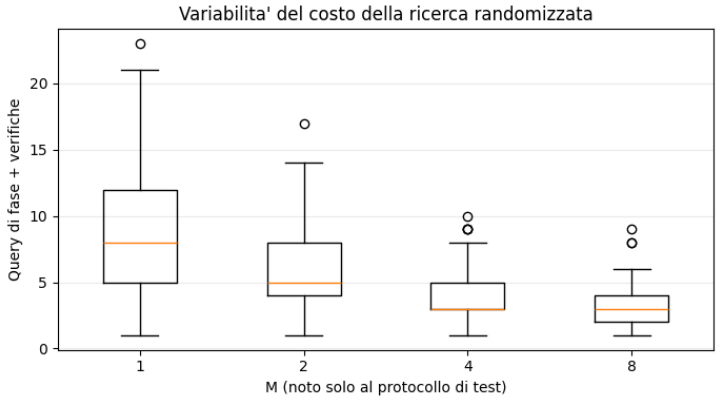


Figura 22: Distribuzione del costo della ricerca randomizzata, calcolato come somma delle chiamate all'oracolo e delle verifiche classiche.

4.5 Simulazione con un fake backend IBM

Nel capitolo 12 del notebook introduco del rumore attraverso i *fake backend* IBM, cioè profili salvati che descrivono le porte, i collegamenti tra qubit e alcuni parametri di errore dei dispositivi. Con `AerSimulator.from_backend` costruisco un simulatore che utilizza questi dati per modellare gli errori delle porte, il rilassamento e gli errori di lettura. Le prove avvengono in locale, senza accedere all'hardware IBM: il modello è un'approssimazione basata sul profilo salvato, non una replica esatta delle condizioni attuali del processore.

Impostazione delle prove. Mantengo fisso il problema: 4 qubit di ricerca, una sola soluzione $|0001\rangle$ e 4096 shot per circuito. Vario il numero di iterazioni da 0 a 6, includendo il massimo ideale a 3 iterazioni e il calo successivo. Ripeto l'esperimento con tre profili:

- `FakeManilaV2`, dispositivo da 5 qubit;
- `FakeNairobiV2`, dispositivo da 7 qubit;
- `FakeMumbaiV2`, dispositivo da 27 qubit.

Il numero di qubit del profilo non modifica la dimensione del problema, che rimane di quattro qubit logici. Per ciascun backend compilo il circuito secondo le porte e i collegamenti disponibili, poi eseguo lo stesso circuito compilato sia con rumore sia senza. In questo modo posso confrontare le frequenze ottenute nelle due condizioni con la probabilità teorica di Grover.

Lettura dei risultati. Le tabelle riportano il successo al variare delle iterazioni. Per ogni profilo, un grafico confronta teoria, simulazione ideale e simulazione rumorosa; un secondo mostra la distribuzione degli esiti a 3 iterazioni, evidenziando la soluzione. Il confronto permette di osservare se l'aggiunta di iterazioni continua a migliorare il risultato oppure se gli errori accumulati riducono il beneficio dell'amplificazione, spostando il massimo rispetto al caso ideale. Le differenze tra i tre profili dipendono sia dai parametri di rumore sia dalla compilazione sulle rispettive topologie: non costituiscono una classifica rigorosa delle prestazioni attuali delle QPU.

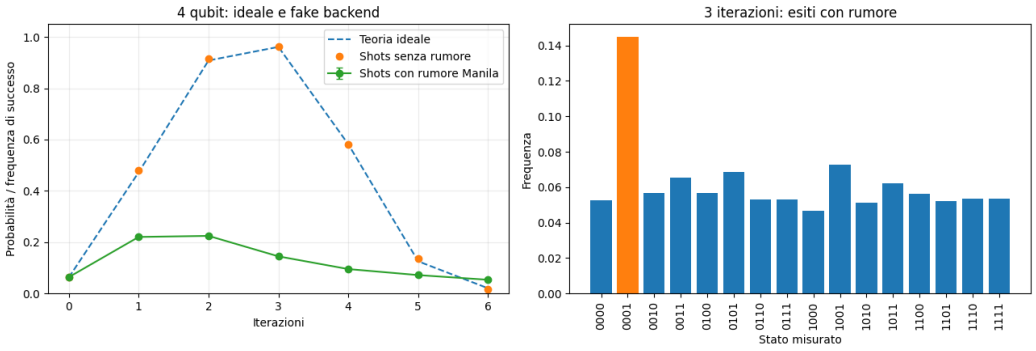


Figura 23: Risultati con `FakeManilaV2`.

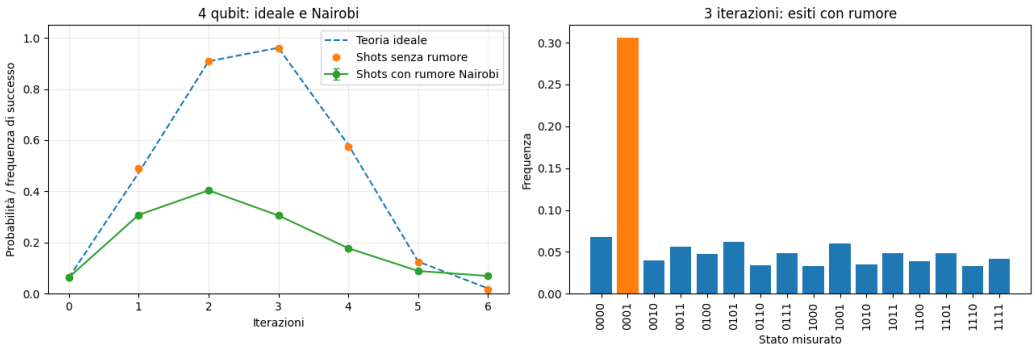


Figura 24: Risultati con `FakeNairobiV2`.

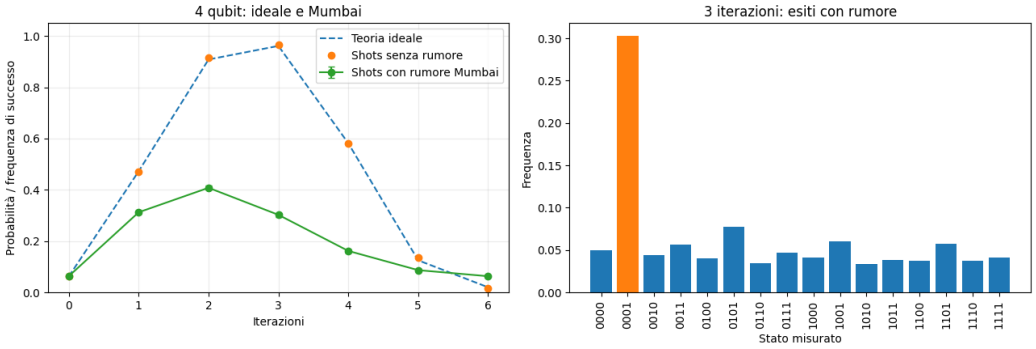


Figura 25: Risultati con `FakeMumbaiV2`.

Resoconto dei risultati.

- **Picco anticipato.** Nei tre profili il massimo misurato si trova a 2 iterazioni, mentre quello ideale è a 3. L'aggiunta della terza iterazione riduce quindi il successo: nel modello rumoroso gli errori accumulati prevalgono sul beneficio atteso dall'ulteriore amplificazione.
- **Manila.** La frequenza massima è circa il 22,46%, contro il 14,48% a 3 iterazioni. Il massimo a 2 iterazioni è comunque molto vicino al 22,05% ottenuto con una sola: questa piccola differenza va letta tenendo conto delle fluttuazioni statistiche.
- **Mumbai e Nairobi.** Raggiungono rispettivamente il 40,82% e il 40,38%, quasi il doppio di Manila. La differenza riflette l'effetto complessivo dei profili di rumore e dei circuiti compilati.

4.6 Esecuzione su hardware IBM reale

Nel capitolo 13 passo dalle simulazioni all'esecuzione su una QPU IBM. Dopo una prima prova con 4 qubit, ho eseguito un confronto con 2, 3, 4 e 5 qubit di ricerca sul backend `ibm_fez`. In ogni caso cerco una sola soluzione, $|00\ldots01\rangle$, con 1024 shot e il numero di iterazioni scelto per il caso ideale: rispettivamente 1, 2, 3 e 4.

Accesso e preparazione. Ho utilizzato un account IBM Quantum Platform con un'istanza Open Plan (ottenuta tramite IBM Cloude, come indicato nel corso), configurando localmente le credenziali tramite `QiskitRuntimeService` con il nome `grover-open`. La chiave API permette l'autenticazione, mentre il CRN identifica l'istanza del servizio. Riutilizzo le funzioni del notebook per costruire i quattro circuiti e li compilo per le porte e la

connettività del backend selezionato. A differenza del test ideale nella base `u`, `cx`, questa compilazione tiene conto di uno specifico dispositivo fisico.

Invio e salvataggio. I quattro circuiti sono inviati con `SamplerV2` in un unico job, così da eseguirli sullo stesso backend. Il notebook salva subito l'identificativo del job e i parametri delle prove in un file JSON, per poter recuperare l'esecuzione senza inviarla nuovamente. Dopo l'attesa in coda e il completamento del job, salvo i conteggi e i riepiloghi in file JSON e CSV nella cartella `results`.

Risultati ottenuti. La tabella riporta i risultati salvati del confronto sul backend `ibm_fez`. La frequenza di successo è il rapporto tra le misure della soluzione e i 1024 shot. Profondità e porte a due qubit si riferiscono ai circuiti compilati associati al job salvato.

	backend	n	k	shots	theoretical_probability	success_probability	depth	two_qubit_gates
0	ibm_fez	2	1	1024	1.000000	0.958984	8	2
1	ibm_fez	3	2	1024	0.945312	0.819336	142	39
2	ibm_fez	4	3	1024	0.961319	0.394531	496	165
3	ibm_fez	5	4	1024	0.999182	0.074219	1405	501

Figura 26: Risultati delle prove su `ibm_fez` con 2, 3, 4 e 5 qubit e 1024 shot per circuito.

Con 2 e 3 qubit la soluzione viene ancora individuata frequentemente; con 4 e soprattutto 5 qubit il successo diminuisce nettamente, nonostante la probabilità ideale rimanga elevata. Parallelamente crescono profondità e numero di porte a due qubit: il circuito richiede più operazioni ed è maggiormente esposto agli errori. Il numero di iterazioni ideale non è quindi necessariamente il migliore su hardware rumoroso; questa prova, questa prova è in linea con i risultati ottenuti in precedenza con i fake backends.

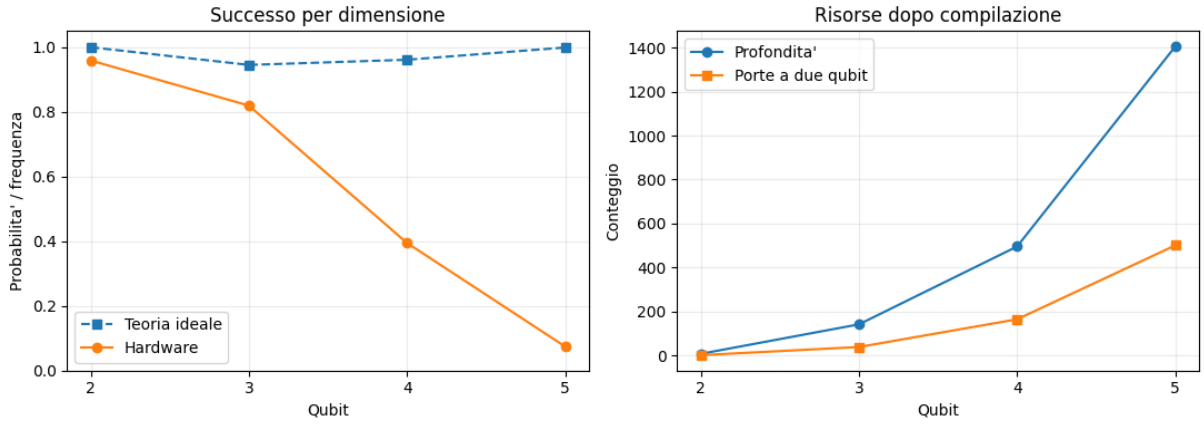


Figura 27: Confronto tra successo ideale e misurato su hardware IBM e crescita delle risorse del circuito compilato al variare dei qubit.